

MCA Pilot Functional Breakdown

CRM Deal Pipeline & Deal Management

Purpose: The core of MCA Pilot is a deal-tracking CRM that follows each merchant deal from initial application through offers and funding. Deals are organized in stages (e.g. New/Incomplete, Active, Funded/Lost) and statuses that reflect their progress ¹ ². A dedicated **Offers Pipeline** helps brokers manage active offers – for example, new lender approvals that need pitching to the merchant, offers where contracts have been requested or sent, etc. – ensuring no opportunity slips through the cracks ³. The pipeline view visually organizes offers by status (New, Pitched, Contracts Requested, Contracts Out, etc.) so reps can push each offer toward funding ³.

User Workflow: Users (brokers/ISOs) create a deal when a new merchant application comes in. This can be done manually via a **New Deal form** or automatically via an intake form integration. The New Deal form captures basic business info (company name, DBA, address, EIN), deal parameters (product sought, etc.), and assignment to team members ⁴ ⁵. The creator assigns **Originators/Closers** (sales reps) to the deal and sets visibility (private to assigned team vs. public to the organization) ⁶ ⁷. Once created, the deal appears in the **Deals list** (searchable, filterable, exportable) ⁸ and is ready for underwriting and submission. As the deal progresses, users update its stage or the system auto-updates it based on events (e.g. marked “Ready to Submit” once all docs are in, or “Funded” once an offer is funded ² ⁹). The system encourages users to keep deals moving – for instance, the Offers Pipeline highlights offers that are awaiting action (on hold, pitched, contracts out) and keeps them in an **Active Offers** view until resolved ³.

Automation: MCA Pilot automates many status changes. For example, an AI “Docs Check” can auto-mark a deal as **Ready to Submit** once the application and latest 3 months of bank statements are uploaded, or flag it as **Missing Documents** if something’s missing ². When a lender’s offer is logged as “approved,” the deal’s offer moves into the Offers Pipeline automatically ¹⁰. If that offer is later marked “Funded,” the system moves the deal into the **Advances** (funded deals) section and out of active pipelines ⁹. The platform also tracks renewal timing – once a deal is funded, it calculates the **paid-in percentage** based on payments, term, and date funded, so the team knows when the merchant might be eligible for a renewal offer ¹¹.

Dependencies: The Deal Pipeline ties into nearly every other module. It depends on **Document Management** (for application docs), **Underwriting** (financial info and deal criteria), and **Lender Submissions** (to generate offers). In turn, changes in those areas update the deal’s status: e.g. the underwriting module flags missing info which sets the deal status to “Incomplete” or “Missing Documents,” which might trigger follow-up emails ². A lender’s offer approval (via email parsing or manual entry) creates an Offer record linked to the deal ¹² ¹³, and funding that offer generates an Advance record. Thus, the CRM pipeline serves as the hub, with other features feeding into or out of it.

Recreating Guidance: To build this from scratch, define a **Deal entity** with fields for all key info (business details, status/stage, assigned users, source, etc.). Implement a **Deal List** view (table of all deals with filters/search ⁸) and a **Deal Detail** page with sub-sections (tabs) for related data (e.g. Documents, Funders,

Offers, Notes). Include a simple **stage progression** model (e.g. Kanban board or status field) that can be updated manually or via triggers. Early on (Phase 1), keep stages simple (e.g. *Application, Submitted, Offer, Funded, Lost*). As you iterate, add a specialized **Offers Pipeline view** (a Kanban board with columns like New, Pitched, Contracts Out, Funded) ³ to visualize offers across all deals. Ensure the database links **deals** → **submissions** → **offers** → **advances** so that updates propagate: for example, if an offer status changes to “funded,” mark the deal as funded and remove it from open pipelines. Basic validations and automation rules (like “if required docs count < 4, status = Missing Docs”) can be implemented via background jobs or triggers. In sum, start with a robust deal object and list view, then layer on automation for status changes and a pipeline UI for offer management.

Document Upload & Management

Purpose: MCA Pilot centralizes all deal files – applications, bank statements, and supporting documents – so brokers no longer juggle email attachments or local folders ¹⁴ ¹⁵. Each deal has a file repository (often in an **Underwrite** tab) where users upload required documents. The system ensures files are organized and standardized: for example, it can auto-rename bank statement files consistently and even apply watermarks or compression so that sharing files with lenders is streamlined ¹⁶ ¹⁷. All files for a deal are accessible in one place and can later be attached to lender submission emails or shared with stakeholders.

User Workflow: In practice, when a new deal is created, the user (or an intake form integration) will attach the **Funding Application** (PDF or e-signed form) and recent **Bank Statements** for the business. Users can drag-and-drop files into the deal's Underwrite/Files section. MCA Pilot provides utilities to manage these files: a one-click **AI Rename** button uses AI to read each bank statement and rename it in a clear format (e.g. `2025-08_Chase_0518.pdf` for August 2025 Chase statement) ¹⁸ ¹⁹. This saves time and imposes a standard naming convention (including year, month, bank name, last 4 of account) for easy reference ²⁰ ²¹. Another feature, **Compress Statements**, reduces PDF file sizes to ensure the total email attachment size stays under common limits (e.g. Gmail's 25MB) ²². The platform may also automatically **watermark** PDF files (especially bank statements) with a company identifier or “for internal use” message – this isn't explicitly described in the docs, but competitors note that MCA Pilot's approach eliminates the need for manual PDF watermarking ²³ ²⁴.

Automation: Document management is closely tied to automation for underwriting and follow-ups. When files are uploaded (especially via an integrated intake form), the system can trigger **AI underwriting analysis** and a **Docs completeness check** automatically ² ²⁵. The **AI Docs Check** feature scans the Application and Bank Statement fields: if the application and the last 3 months of statements are present, the deal status auto-updates to “Ready to Submit.” If anything is missing (say one month of statements or the signed app), the deal is marked “Missing Documents” and the system notes exactly which items are missing ² ²⁶. Those missing document names (e.g. “Funding Application” or “June 2025 Statement”) are stored in a field and used to inform the team **and** the merchant. For example, an automated **Follow-Up email** can pull in the `{missing_docs}` variable to list which files are needed and include a secure upload link for the merchant to submit them ²⁷ ²⁸. The upload link (provided via an automation workflow) feeds directly back into the deal's documents, so when the merchant uploads the missing files, the deal is updated in real-time ²⁹ ³⁰. Additionally, MCA Pilot's **Fast AI Underwriting** function will read the bank statements and extract financial metrics once they're uploaded – this is covered in the underwriting section but is initiated by the presence of these files ³¹ ³².

Dependencies: Document management relies on having a defined list of **required documents** for each deal. The platform likely considers a standard set (e.g. application form, last 3 months' banks by default) but may allow configuration of what's "required." This module feeds directly into **Underwriting** (AI can't analyze statements you haven't uploaded) and into **Submissions** (files from the deal's repository are attached when emailing lenders). The completeness status also feeds into **Follow-Up automation** (triggering reminders for missing items) and influences deal status (as described above). In turn, the intake **Forms integration** or **API** populates this repository: if a JotForm or DocuSign is integrated, it will drop the PDF and any uploaded statements into the deal automatically, kickstarting the underwriting workflows ²⁵. There is also a dependency on **Email Integration** if large files need to be compressed before sending – the system's file compression ensures that when the Email Send module picks up these files to send to lenders, it won't hit size limits ²².

Recreating Guidance: To build a similar document module, create a **file storage component** linked to each deal (e.g. a cloud storage bucket or database of file records). Provide an **upload interface** on the deal detail page for users to add files under specific categories (Application, Bank Statements, Other Docs). Implement background jobs for enhancements: e.g., after upload, trigger a **rename function** that uses either filename heuristics or an AI text-reading service to identify bank name, date, account number in statements ³³. Implement a naming schema (YYYY-MM_Bank_####) and apply it to those files. Also set up a **file compression utility** (there are open-source PDF compressors or use an API) to optimize PDFs once uploaded. For checking completeness, define what constitutes a complete package (you might start with simple rules: e.g. if 1 application file and ≥ 3 bank statement files present, mark complete). More advanced: incorporate a dictionary of months in statements to ensure you have the latest three months. You'd also implement a **Missing Docs tracker**: e.g., if not complete, determine which ones are missing (perhaps store required doc names in an array for the deal, then subtract uploaded types). This can populate text that can be merged into emails (like "Missing June 2025 Statement"). To allow merchants to upload, you can generate a unique **upload link or portal** – initially, a simple approach could be an upload form at a fixed URL with a query token identifying the deal. The key is to automatically associate any file uploaded via that link to the correct deal and update the status accordingly. By Phase 2, aim to integrate e-signature or form providers: for example, use webhooks or API from DocuSign/JotForm to receive completed applications and files and attach them to deals (MCA Pilot offers to "map all the fields and sync your apps into the CRM" for whatever intake system you use ³⁴, so you'd replicate this by writing mapping scripts for each integration or by using middleware like Zapier).

Underwriting & Deal Evaluation

Purpose: This module helps brokers quickly evaluate the financial health of a merchant and prepare the deal for funding offers. It combines **AI-driven analysis** of financial documents with structured data fields for credit criteria. The goal is to replicate what a human underwriter would glean (monthly revenue, average daily balance, NSF count, existing advance positions, etc.) and make that data instantly available for decision-making and **lender matching** ¹⁷ ³⁵. In short, MCA Pilot's underwriting features aim to turn raw application data into actionable intelligence: it extracts key metrics from bank statements, checks the deal against various lender criteria, and even calculates things like estimated payment capabilities or risk flags, so brokers can target the right funders and offers for the merchant.

User Workflow: After uploading the merchant's bank statements and application, the user clicks a one-button **"AI Underwrite"** (labeled *Fast AI Underwriting*) in the deal's Underwrite tab ³¹ ³². The system then analyzes the statements – using AI/OCR to read transaction data – and extracts important financial figures

and insights. For example, MCA Pilot's AI will pull out: the average monthly deposit volume, average daily balance, number of NSF's (bounced checks) in each month, number of negative balance days, any unusual large deposits, etc., as well as attempt to detect **existing MCA positions** (e.g. identifying regular ACH debits to known funders) ³¹. The result of this analysis is typically stored in the deal's **Criteria/Financials fields**. The broker can review these in a structured format (perhaps under a "Financials" or "Criteria" tab on the deal). Simultaneously, the system can compare these values to typical funder requirements – highlighting, for instance, if the merchant has more NSF's than some funders allow, or if the revenue is too low for certain lenders.

MCA Pilot also provides an **AI Application Scan** feature for data entry: instead of typing all the merchant's details, the user can upload a filled PDF application form and the system will auto-extract the fields to create or update the deal ³⁶ ³⁷. This means business owner info, business start date, credit scores (if present on app), etc., are parsed and filled in. With that, along with bank statement analysis, the deal's profile is largely complete without manual data entry.

Automation: The underwriting module automates both **data extraction** and **decision support**. The *Fast AI Underwriting* workflow populates hidden fields like "Avg. Monthly Revenue = \$X" or "NSF's (last 3 mo) = Y" which can then be used to auto-qualify or disqualify funders in the matching engine ³⁸ ³⁹. If the AI detects, say, that the merchant currently has two other advances, it might set the deal's **Position** field to "3rd" (because a new advance would be third position). All these criteria – FICO score, Time in Business, Annual Revenue (converted to monthly), industry type, state, positions, and any special cases (e.g. construction or trucking industry specifics) – feed into the **Auto-Match** feature (described in the next section) ⁴⁰ ⁴¹. Another aspect of automation in underwriting is **document checking** (as covered earlier): the system verifies if required docs are present and updates statuses and notes accordingly ².

Notably, MCA Pilot's AI can also parse **lender criteria sheets** to aid underwriting on the funder side. The **AI Scan Criteria** tool allows the user to upload a PDF of a lender's guidelines (their underwriting criteria) and automatically fill the lender's profile with those criteria ⁴² ⁴³. This is a reverse side of underwriting: instead of extracting merchant data, it extracts funder data (min/max credit scores, funding limits, prohibited industries, etc.) and updates the system. By automating this, the platform ensures that the matching of deals to funders is as accurate and up-to-date as possible without hours of manual data entry for each lender ⁴⁴ ⁴⁵.

Dependencies: The Underwriting module depends heavily on **Document Management** – without an application or bank statements, the AI has nothing to analyze. It also relies on the **Funder Management** data (criteria) to determine what metrics to extract and compare. There is a dependency on having an AI/OCR service or model trained for bank statements; MCA Pilot likely uses a combination of OCR and predefined logic or ML models to detect patterns (e.g., identify a series of similar debits as MCA payments, or read the summary section of a bank PDF for total deposits/NSF's). Underwriting results then directly influence **Lender Matching** (which funders show up as qualified) and can even trigger workflow changes (e.g., if AI finds the deal is very strong or very weak, perhaps custom flags or different default statuses could be applied). Additionally, underwriting outputs feed the **Follow-Up** module: for instance, if the merchant was declined by all funders due to low revenue or high NSF's, the broker might schedule a follow-up for a few months later after the merchant's finances improve. MCA Pilot's design pushes toward an "autopilot" state ⁴⁶, so each piece of data extracted is meant to reduce human intervention in deciding next steps.

Recreating Guidance: To build underwriting automation, start by identifying the key data points you need from merchant inputs. Phase 1 might not include full AI—begin with structured **form fields** for things like monthly revenue, FICO, etc., that a user can input or import from a form. In parallel, you can use third-party OCR APIs (e.g. Azure Form Recognizer, Amazon Textract) to extract text from bank statement PDFs. Then write parsing logic to find relevant figures (search for keywords like “Total Deposits” or specific patterns like dates and balances). As an MVP, even a simplified parser that looks at summary pages of bank statements could populate: e.g., total credits for each month, count of NSF fees (by searching for “NSF” occurrences). Over time (Phase 3), incorporate a trained AI model or a service specialized in financial statement analysis to improve accuracy ¹⁷ ⁴⁷ . Also create fields on the **Deal** for each underwriting criterion (Time in Biz, Revenue, Positions, etc.) and for each, define the logic or source: some come from the app form (TIB from business start date, FICO from credit pull or form input), others from statement analysis (revenue, NSFs). Ensure you have a UI in the **deal’s Criteria tab** to display these and allow manual override if needed. Next, integrate **funder criteria**: design a **Funder entity** that holds all the same criteria fields. For automation, implement an **“AI Scan Criteria”** function – you can use OCR on a PDF of guidelines and look for known terms (e.g. “Minimum FICO” followed by a number, or lists of restricted industries). This is complex, so an alternative is to crowdsource initial data entry and use AI for updates. However, since we want autopilot, you’d gradually add parsing rules. The key is to store data uniformly (e.g. in numeric form, “9999999” as placeholder for “no max” as MCA Pilot does ⁴⁸ ⁴⁹) so your matching algorithm can simply compare deal vs. lender criteria. Lastly, implement triggers: e.g., after running AI underwriting, call a function to **auto-update deal status** (if all good, “Ready to Submit”; if issues, maybe “Needs Review” or specific flags). By Phase 3, you can add a recommendation engine: for instance, highlight if the deal meets *premium* criteria (could get better rates) or if it’s borderline, etc., to guide human users.

Lender Management & Auto-Matching

Purpose: This module manages the broker’s network of funding partners (lenders or “funders”) and streamlines the process of selecting the right ones for each deal. It serves two key functions: (1) A **Funder Database** where the broker stores each lender’s contact info, submission emails/portal info, and detailed funding criteria; (2) an **Auto-Match engine** that uses those criteria (and the deal’s underwriting data) to filter and recommend the best funders for a given deal ⁵⁰ ³⁵ . By having all lender profiles in one place, MCA Pilot eliminates the need for brokers to remember which lenders handle which scenarios – the system can instantly tell you, for example, that Deal X with 6 months in business and a 600 FICO is eligible for Lenders A, B, and C, but not D (perhaps D needs 2+ years TIB, etc.) ⁵¹ ³⁸ .

User Workflow: In MCA Pilot, users set up their funders usually during onboarding. **Adding a Funder** involves inputting basic fields (company name, contact person, submission email address or portal link) and then entering the lender’s guidelines. The system likely provides a form or spreadsheet-like interface under a **Funders section** (or Settings > Funders) to input criteria such as: Minimum FICO, Min/Max Time in Business, Min/Max Funding Amount, Max positions, restricted industries, restricted states, products offered (e.g. do they do 2nd positions? do they offer LOCs?), typical terms, etc. ⁴⁰ ⁴¹ . This can be time-consuming, so the **AI Criteria Scan** feature is a big help – the user can upload the lender’s PDF guideline, and the system auto-fills those fields ⁴² ⁴³ . Once funders are set up, the broker can group them by **Tags** or grades. For example, the user might tag certain funders as “A Paper” or “High Risk” or “Preferred” to quickly filter them later ⁵² ⁵³ . They can also assign a “paper grade” or similar categorical label to each (A, B, C paper), depending on the platform’s design ⁵⁴ .

When working on a deal, the user goes to the deal's **Funders tab**. Here MCA Pilot will display a list of **Qualified Funders** automatically, by comparing the deal's profile to each lender's criteria ⁵¹ ³⁵. For instance, if the merchant's FICO is 550 and annual revenue \$500k, the system might filter out any lender requiring 600+ FICO or \$1M revenue – those will appear either not at all or under an "Unqualified/Filtered Out" list with reasons ⁵⁵ ⁵⁶. The user can review the **Mismatch Reasons** for any funder that was filtered out by hovering/clicking – e.g. it might say "FICO too low (550 < 600)" or "Industry restricted: Construction" ⁵⁷. The user can then choose from the **Qualified funders** which ones to submit to. MCA Pilot allows bulk actions: the user can multi-select funders (individually or by filtering by tag/grade) and hit "Add to Queue" ⁵² ⁵⁸. This populates a submission queue with those lenders. From there, a single **Submit** action (described in the next section) will send out to all in the queue ⁵⁹ ⁶⁰.

If the broker prefers not to rely on the criteria matching, they can use their own groupings: e.g., filter the Funders list by a **Tag** like "Stacked Paper" for a highly indebted merchant, and then bulk-add those. The system supports this flexibility: tags, paper grades, or the pure criteria match can all be used in combination ⁶¹ ⁵⁴. The end result is that within seconds, the rep has a curated list of appropriate lenders for the deal, rather than manually cross-referencing guidelines.

Automation: Once set up, the Funder criteria database makes the matching automatic. Whenever a deal's data is updated (say the merchant's credit score is added), the **Qualified Funders list** on that deal will dynamically update ⁶². The platform likely runs the matching algorithm on loading the Funders tab or when data changes. Moreover, if a required field for matching is missing, the system informs the user ("a required field is missing" for either the deal or the funder) ⁶². This prompts the user to fill any blank (for example, if no industry is set on the deal, some funder filtering can't happen, so the system will exclude those funders until the field is provided). On the funder side, **AI Criteria Scan** automates maintenance of the database – whenever a lender updates their terms, the broker can upload the new PDF and the system will extract changes (with safeguards to not overwrite certain fields like contact info unless blank) ⁶³ ⁴⁸.

Another automation is **Portal vs. Email handling**. Some funders require submissions via their web portal instead of email. MCA Pilot lets the user mark such funders appropriately, and then when the user hits submit, it will *not* send an email but instead put that submission in a special "Portal Submissions" section for tracking ⁶⁴ ⁶⁵. This way, the broker is reminded to go to that portal and submit manually, but the system still tracks it. Thus, the system automates the *filtering* of how submissions are sent based on lender preferences.

Dependencies: Lender Management depends on **Underwriting/Deal Data** to function – the auto-match algorithm needs both sides' data. If either the deal's or funder's criteria are incomplete, the matching could fail or omit funders ⁶² ⁶⁶. This encourages users to use the AI extraction for both ends (deal app scan and funder criteria scan). It's also tied to the **Submission module**: funder profiles include the **contact emails, CC rules, or API integration keys** needed to actually submit deals. For example, a funder record will have an email address to send deals to, and possibly an indicator if they accept multiple offers or if they reply in thread or new email – this can influence how the email content is structured (some systems require specific subject line formats; MCA Pilot even has a feature for funder-specific subject line prefixes to automatically label threads ⁶⁷ ⁶⁸). Additionally, if the platform integrates directly with certain funders via API, the funder record holds the credentials or endpoints for that (e.g. Lendr, Bitty are mentioned as API integrations ⁶⁹ ⁷⁰).

Recreating Guidance: Building a Funder Management module means creating a **Lender/Funder entity** with fields for criteria and contact info. Start with the basics: name, contact email(s), phone, and a free-text “notes” for any special submission instructions. Next, design the schema for criteria: you might have fields like `min_credit_score`, `min_time_in_business_months`, `max_time_in_business_months`, `min_annual_revenue`, `max_annual_revenue`, `max_positions`, `restricted_industries` (likely a multi-select or tag list), `restricted_states`, etc., based on common MCA lender guidelines ⁴⁰ ⁷¹. Also include flags for `offers_products` (e.g. does this funder do 2nd positions, offer LOCs, purchase orders, etc.) and any commission or fee structures if relevant. Implement a **function to evaluate match**: given a deal and a funder’s criteria, return true/false and reasons. Early on, this can be simple comparisons (deal.FICO >= funder.min_FICO, etc.). Build a UI on the deal page that lists all funders with a filter for “Qualified” vs “Unqualified”. This likely involves running each funder through the match function and categorizing the output. Displaying mismatch reasons can be done by storing messages from each failed condition (e.g. if deal.FICO < min_FICO, add reason “FICO below minimum”). For filtering and grouping, implement a **Tag system**: allow tagging funders with arbitrary labels and storing those. On the deal’s funder selection UI, allow filtering by tag or a dropdown for paper grade. This is essentially just a filter on the funder list UI (e.g. show only funders where “preferred” tag is present).

For automation, add a feature to **bulk-add funders** to a “submission queue.” In your data model, perhaps have a `deal_submissions` table that records which funders are attached to a deal before sending. The user action “Add to Queue” can create those records for selected funders, and a subsequent “Submit All” will iterate through them. To handle **portal vs email**, add a field like `submission_method` on funder (Email vs Portal vs API). Then, your submission sending logic can route accordingly – emails go out via SMTP, portal ones perhaps just mark an entry “to do”, and API ones trigger an API call.

Finally, to assist with **criteria input**, consider integrating an OCR or text processing for guidelines. For MVP, you might skip AI and let users fill forms. But long-term (Phase 3), you can implement an **“Upload Guidelines”** function. This might use NLP to find key terms as mentioned. For example, you could use regex for patterns like “Minimum FICO: ###” or look for a list of industries under a “Restricted Industries” heading, etc. This is advanced, but even partially automating (like extracting numbers and suggesting them in the UI for confirmation) would add value. Also, maintain good UX: allow users to override or edit any AI-filled criterion, and crucially, **don’t overwrite manual entries without confirmation** (MCA Pilot’s approach is to only fill blanks with AI to avoid erasing custom inputs ⁴⁸ ⁷²).

One-Click Deal Submission to Lenders

Purpose: This feature automates the actual deal submission process – sending the merchant’s application package out to chosen lenders – with a single action. Traditionally, a broker would craft separate emails to each funder, attach the application and statements, and perhaps fill out a cover email with deal details. MCA Pilot streamlines this by generating those emails (or API submissions) automatically with standardized content and tracking each “submission” record in the system ⁷³ ⁷⁴. The goal is to **“sub out” deals in seconds instead of hours** ⁷⁵ ⁷⁶, ensuring that every relevant lender gets the info at the same time and no submission falls through the cracks.

User Workflow: After selecting the target funders in the Funders tab (adding them to the queue as described above), the user clicks the **Submit** button on the deal. A **Submit Deal dialog** appears, where the user chooses the **Sender email account** to send from (this defaults to their configured email, but they can change it each time) ⁷⁷ ⁷⁸. They verify that all attachments are in order (the dialog likely shows which files

will be attached – usually the application PDF and bank statements) ⁷⁹. They then confirm, and MCA Pilot sends out a separate email to each selected funder automatically ⁸⁰. Each email is personalized with that funder's name, and possibly certain variables or formatting that funder prefers (e.g. including the merchant's name in the subject, or a prefix like "Submission:" – the platform supports funder-specific subject prefixes to align with funder requirements ⁶⁷). The content of the email often includes a summary of the deal (business name, requested amount, perhaps a blurb like "Please see attached application and 3 months statements for [Business] seeking \$X working capital."). MCA Pilot likely has a default email template so that brokers don't need to write anything for each submission – it's automatically generated and sent via the connected SMTP/Email account of the user ⁷³.

Once sent, the system creates a **Submission record** for each funder emailed. Users can view submissions either on the deal page (an **Activity or Submissions section** listing all funders it was sent to) or in a global **Submissions list** in the sidebar ⁸¹ ⁸². The submission record tracks the status (Sent, Pending, Responded, etc.) and stores the timestamp, sender, and a link to view the content or any error messages if the email failed ⁸³ ⁸⁴. For example, in the deal's Submissions table, you might see: Funder = "Titan Funding", Status = "sent" (orange label), Submitted By = John Doe, Submitted = "5 minutes ago" ⁸⁵. Another might show Status = "portal" (purple label) indicating it wasn't emailed but needs portal action ⁸⁵. Each record can later be updated (either manually or by the AI parser) to statuses like **declined** (if the lender emailed back a decline), **approved** (if an offer came), **funded** (if that submission led to funding), etc. ⁸⁶ ⁸⁷.

MCA Pilot's one-click submission covers not just email but also includes direct **API integrations** for certain lenders. For example, if Lendr or Bitty is integrated via API, when the user hits submit for those, MCA Pilot will send the deal data through the API instead of email ⁶⁹. This happens behind the scenes but from the user's perspective it's the same one-click action – they may just see the submission status or notes indicating it was via API.

Automation & Tracking: The moment the user clicks submit, MCA Pilot automates everything: it chooses the correct template, attaches files, sends emails (through the user's email account via SMTP or API), and logs each attempt. It also handles any failures – e.g., if an email is not sent successfully (due to wrong password or attachment too large), the submission status will show **Errored** and allow the user to view the error message ⁸³ ⁸⁴. Common errors like SMTP authentication issues or "message too big" are caught, and MCA Pilot provides guidance to fix them (such as using an app-specific password for Gmail, or compressing files if over size limit) ⁸⁴ ⁸⁸.

The system also automates CC or BCC rules if configured. The user's profile might allow default CC's (for instance, always CC a manager or the rep's own address). The platform mentioned you can choose who to CC on each submission and set defaults ⁸⁹. This means the content generation includes those addresses automatically, saving the step of adding colleagues or "iso@mycompany" on every email.

Each submission entry then awaits a response. In the meantime, MCA Pilot provides shortcuts like an **"Open in Gmail"** button on the deal or submission, which opens that email thread in the user's Gmail if needed ⁹⁰ ⁹¹. This quality-of-life feature shows the integration depth: the system logs the message-ID or link to the sent email, so the user can quickly jump to it if they want to follow up or verify it was sent.

Dependencies: One-click submission relies on the **Email Integration (Sender Accounts)** being configured. Each user must connect at least one email (via SMTP or OAuth) to use as the "From" address ⁹² ⁹³.

Without that, the feature can't send emails on the user's behalf. The content of submissions depends on templates that likely pull from **deal data fields** (so it depends on having an email template with placeholders like {business_name}, {requested_amount}, etc.). It also depends on the Document module for attachments and the Funder records for knowing *where* to send each email (the "To" address comes from the funder profile). If using APIs, it depends on the **Funder API integration module** (for each integrated lender, the system must map the deal fields to the lender's API fields; the **AI Application Scan** ensures those fields are populated so the API call can include a full app ⁹⁴ ³⁷). The Submissions tracking is also linked to the **AI Email Parsing** module – as we'll see, when a lender replies, the parser updates the corresponding submission record. In essence, the one-click submission sets the stage for all subsequent automation: it initiates the lender responses that AI will parse, and populates the pipeline of offers.

Recreating Guidance: To implement one-click submissions, you need a few components: (1) **Email templating**, (2) **Email sending via user accounts**, (3) **Submission logging**. First, set up the ability for users to connect an SMTP or use OAuth for Gmail/Outlook. You might integrate a library or service to send emails on behalf of users (make sure to store credentials securely or use OAuth tokens). Provide a UI for users to add "Sender" profiles, including fields like SMTP host, port, username, password, from-name, etc. (MCA Pilot's docs detail how users add an SMTP sender and verify it ⁹⁵ ⁹⁶). Next, create an **email template** for submissions. In early stages, this can be a static template all funders get. Eventually, allow customization per funder or globally. The template should pull in deal fields (like business name, contact info for the merchant, etc.) and perhaps a list of attached filenames.

When the user hits "Submit Deal", your backend should generate one email per selected funder. Use the funder's email address as **to**, the user's selected Sender as the **from**. If the funder is marked as portal, skip sending and instead log a submission with status "portal" (and possibly prompt the user with instructions like "Please upload to portal and then mark as done"). For each email, attach the deal's documents (the ones flagged as application/banks). Send the email and capture the result: if success, mark submission status "sent"; if failure, mark "error" and store the error text. All this should happen asynchronously (so consider a job queue to send each email, especially if a user selects 10 funders at once). After sending, insert a **Submission record** linking the deal and funder, with status, timestamps, etc. Also store identifiers: for example, message-id of the email or any thread token, to use later for matching replies.

Implement a **Submissions List UI**: on the deal page, show a table of submissions (funder name, status, response summary, who sent, when sent) ⁸⁶ ⁸⁵ . Also consider a global "Submissions" screen where users can monitor all deals' submissions in one place ⁸¹ – this is useful for managers to see everything in the pipeline.

Finally, handle email sending edge cases: incorporate a check for attachment sizes (sum of file sizes) and use or suggest compression if too large (MIME encoding overhead means ~25MB limit for Gmail, so possibly auto-compress if >18MB as MCA Pilot does ⁸⁸ ⁹⁷). Also implement CC/BCC defaults: could be user-specific settings (e.g. always CC my manager) and allow override in the submit dialog. This level of polish can come in Phase 2. Phase 1 can simply send to one primary recipient per funder and rely on the user's email Sent folder if they need to see what was sent.

Email & Inbox Automation (AI Response Parsing)

Purpose: One of MCA Pilot's standout features is **AI Inbox Processing** – it monitors the user's (or team's) email for lender responses and automatically updates the system with those outcomes ⁹⁸ ⁹⁹. This turns what used to be a manual task (reading each lender's email reply and then recording in the CRM whether the deal was approved, declined, or needs more info) into an automated workflow. The benefit is huge: brokers don't miss responses, don't waste time on data entry, and get real-time status updates on their deals without constant inbox checking ¹⁰⁰ ⁴⁶. In short, this feature moves the "management" part of deals closer to autopilot by handling the back-and-forth communications with funders.

How It Works for Users: The user first enables this feature by configuring **IMAP access** for their email account within MCA Pilot ¹⁰¹ ¹⁰². Essentially, the system needs read-access to the inbox. Users likely go to a settings area (perhaps under their Sender or a dedicated Email Integration page) and provide IMAP host, port, and an app-specific password (for Gmail, as detailed in docs, or OAuth for Microsoft accounts) ¹⁰³ ⁹³. Once set up, the AI processing service runs periodically (MCA Pilot mentions every 15 minutes) to fetch new emails from the inbox ¹⁰¹. It filters those emails: only messages from known lender domains (those matching any funder's email domain in the system) are considered ¹⁰⁴ ¹⁰⁵. This ensures it ignores unrelated emails. If a lender email is found, the system tries to match it to the relevant **Submission** record. If the lender replied directly to the email thread MCA Pilot started, it's easy – the reply will have a reference (In-Reply-To or same subject) to tie it to a submission ¹⁰⁶. If the lender starts a new email thread (which sometimes happens), the AI will use its content to determine which deal it belongs to (likely by looking for the merchant name or other clues) ¹⁰⁴ ¹⁰².

Once it identifies the submission, the AI will **classify the email** into one of three broad outcomes: **approved**, **declined**, or **pending (needs more info)** ¹² ¹⁰⁷. This classification is done with natural language processing on the email text. For example, phrases like "we can offer" or an attached approval document would trigger "approved," while "unable to fund" or "decline" triggers "declined," and "need the following stips" might trigger "pending/conditions." After classification, the AI **extracts key details** from the email. For an **approved offer**, it pulls out terms: the funding amount, factor or rate (sell rate), term length, payment frequency (daily/weekly/monthly), any fees or commission points, perhaps the buy rate or the offer's payback amount, and any listed stipulations (conditions) required ¹² ¹⁰⁸. It might also catch an **offer link** if provided (some lenders include a link or PDF to the contract). For a **decline**, it will look for the reason text (e.g. "declined due to credit score" – it might capture "credit score too low" as the reason) ¹⁰⁷ ¹⁰⁹. For a **pending/need info**, it will try to list what additional information or documents the lender requested (e.g. "please provide January bank statement" or "need landlord contact" – collating these into a task list) ¹¹⁰.

After extracting, MCA Pilot will **take actions in the system automatically**:

- If **approved**, it will **create an Offer record** under that deal, populating all the terms it parsed ¹² ¹¹¹. This is as if the user manually clicked "Mark as approved" and filled the form, but it's done for them. The submission's status also changes to "approved" (with perhaps a green label) ¹³. The newly created offer will then appear in the **Offers Pipeline** (likely in the "New" column since it's an offer that needs to be pitched to the merchant) ¹¹².
- If **declined**, it simply updates that submission's status to "declined" (often with a red label) and logs the reason text somewhere (possibly in a Response field or note for that submission) ¹⁰⁷ ¹¹³. This way the deal's record reflects that outcome without user intervention.
- If **conditions requested** (pending), it updates submission status to "pending" and, very helpfully, it will

create tasks or notes for each required stipulation ¹¹⁰ ¹¹⁴ . For example, it might create tasks under the deal: "Obtain updated bank statement" or "Get landlord reference letter," etc., as listed in the email. This gives the broker a to-do list to move the deal forward.

All these happen behind the scenes. Next time the user opens the deal, they might find "Evergreen Funding – Declined (Reason: negatives)" already marked, or a new \$50k offer from "SwiftBiz Capital" already waiting in the Offers tab, without them reading any email ⁸⁵ . This automation can drastically reduce turnaround time; brokers can focus on talking to merchants and closing, rather than clerical email updates.

Dependencies: The email parsing feature depends on the **Email Integration (IMAP)** being set up and on having a consistent method to identify funder emails. Each Funder in the system likely has a domain or specific email pattern associated (like *@funderDomain.com*) *which the parser uses to know an email is from a funder* ¹⁰⁴ . *It also depends on the Submission records existing and being up-to-date (with at least which funder was sent and possibly merchant name) to match replies. The quality of this feature relies on good NLP – it likely uses an AI/ML model trained on many lender emails to interpret various formats of offers and declines (since each lender writes differently). If an email is very unstructured or the AI is unsure, presumably it would not act or might leave it for human review. The system's ability to extract terms correctly is crucial; it probably has a library of typical terms (like identifying something like "1.49 buy rate" or "12-month term" in the text). The Tasks/Follow-up* system is leveraged for pending items, showing a dependency there (the AI uses the task creation feature to assign out what to collect). Moreover, this automation requires trust – thus MCA Pilot initially labeled it beta* ¹¹⁵ . A brokerage implementing this should monitor for mistakes (so there's a dependency on user oversight to correct any erroneous AI entry).

Finally, if using Microsoft Outlook or other providers, there are specifics: MCA Pilot provides guidance for Microsoft accounts (likely using OAuth and dealing with throttling issues) ¹¹⁶ . So the integration approach might differ per email provider (IMAP vs Graph API), which adds complexity in development.

Recreating Guidance: Building an AI email parser is one of the most advanced parts – likely a Phase 3 feature after the core CRM is stable. To implement it, break it down:

- **Email fetching:** Set up an IMAP client or email API client that can periodically read inbound messages from the user's mailbox. You could allow the user to provide IMAP credentials (less secure but straightforward) or integrate OAuth for Gmail/O365 (more complex but secure). Start by fetching emails from a specific folder (maybe a dedicated folder or just Inbox) and filtering on new, unread messages.

- **Matching to deal:** Use simple rules first: if the email is a reply to a sent submission, you could include a hidden reference in your original submission emails (e.g., include the deal ID or submission ID in the email headers or even in the footer). For instance, some systems add a unique ID in the footer like "Deal ID: 12345" which isn't seen by humans much but can be picked up. If that's there, matching is trivial. If not, use the **sender's email/domain** and the context: find submissions where funder's domain matches and that are still pending a response. If the email mentions the merchant or deal name, that can confirm the match. Develop a heuristic: e.g., for each new email from a known funder domain, check for any active submission for that funder in the last X days; if only one, match it. If multiple, perhaps parse the email for the merchant's name and match to deal.

- **Classification & Extraction:** This is where NLP is needed. Early approach: use keyword heuristics. For example, if email body contains "approved" or "offer" or "approved for" – classify as approval. If contains "decline", "unable to fund", "regret to inform" – classify as decline. If contains "need" or "require" and asks for things – classify as needs info. Then extract specifics: for approvals, look for patterns like "\$" for amounts, terms like "months" or "payments" etc. A simple way is to leverage existing AI: connect to an NLP service or

even use GPT-based parsing by feeding the email text (redact sensitive info) and asking it to output a JSON of terms (funding amount, term, rate, etc.). That could be effective but consider privacy and cost – perhaps fine-tune a smaller model later. Also, many funders attach formal offer sheets; parsing PDF attachments might be another step (though MCA Pilot's description suggests they focus on email text content). For declines, capturing any sentence after “because” or “due to” could get the reason. For pending, a bullet list or sentence with requested items is likely present; you could split by line breaks or common phrases like “need” or “please provide”. Over time, training a model on actual emails would be ideal (MCA Pilot likely did this over many examples).

- **System updates:** Once you have structured data from the email, update your records. If approved, create an **Offer entry** (with fields for amount, term, rate, etc.) and link it to the corresponding submission/deal ¹¹¹. If your system marks offers vs submissions separately, make sure to also set `submission.status = “approved”` and link the offer to it. If declined, mark `submission.status = “declined”` and store the reason (maybe in a `response_note` field). If conditions, mark `submission.status = “pending”` and create individual **Task records** for each requested item. Also, consider notifying the deal owner – maybe trigger an in-app notification or email to the rep: “Lender X responded with an offer of \$Y” or “Deal declined by Lender X.” This keeps the team in the loop.

To ensure this runs reliably, set up a schedule (e.g. a cron job every 10-15 minutes, or an email webhook if possible). Provide a UI toggle or indicator so users know it's on (and can disable it if needed). Phase 1 of this feature could start with just capturing **basic status** (approved/decline) and perhaps not auto-creating the full offer until later. But given the time-saving, even a minimal version is useful. Just be mindful to test thoroughly – misclassification of an approval vs decline could be problematic. Start with conservative rules and let ambiguous cases default to “needs review” for human handling.

Example in action: After implementation, a scenario will look like: Broker hits submit to 5 lenders. Later that day, 3 emails come in – one says “Approved for \$50k @1.49 12mo”, second says “Declined due to NSF,” third says “Need April statement and proof of ownership.” Your automation picks them up: it creates an offer record for \$50k (and maybe notifies the rep), marks the second submission declined with reason, and for the third, marks pending and creates two tasks (“Upload April statement”, “Upload proof of ownership”). The rep logs in and sees all this already done – they can quickly act on tasks or call the client with the \$50k offer. This is the kind of efficiency MCA Pilot aims for ⁴⁶.

FUNDER	STATUS	RESPONSE	SUBMITTED BY	SUBMITTED
Titan Merchant Funding	sent	--	Victor Fteha	7 days ago
Evergreen Business Funding	declined	Negatives	Victor Fteha	22 days ago
SwiftBiz Capital	funded	--	Victor Fteha	22 days ago
Summit Business Funding	portal	--	Victor Fteha	25 days ago
NovaFunding Partners	sent	--	Victor Fteha	27 days ago

Example of a deal's Submissions tracking: each lender submission is listed with its status (e.g. sent, declined, funded, portal). Responses like "Negatives" are logged for declines. AI parsing will update these statuses and add offers automatically when lenders reply ¹² ¹³ .

Offer Management & Deal Closing

Purpose: Once lenders start responding with offers (approvals), those offers need to be managed to convert them into a funded deal. MCA Pilot provides tools to organize multiple offers per deal, present them to the merchant, and track the merchant's decision process (Pitched, Docs Out, Funded, etc.) ³ . Essentially, this is the CRM's deal-closing workflow, ensuring that every approved offer is followed up on and either funded or lost. The Offer Management module covers recording offer details, updating their status as the merchant moves forward, and finally converting an accepted offer into a **Funded Advance** record with commissions, etc.

User Workflow: When an offer is created (either via AI or manually by marking a submission as approved), it appears under the deal's **Offers tab** and in the global **Offers Pipeline** view ⁸⁷ ¹¹⁷ . Each offer record contains the lender who approved it and all the terms (amount, factor/rate, term length, payment frequency, any fees, commission points, etc.) ¹¹⁸ ¹¹⁹ . Users can click into an offer to see full details or make edits if needed (for instance, if a lender revises an amount or if an error was corrected). At this stage, the broker will typically communicate these offers to the merchant. MCA Pilot facilitates this with automated follow-ups: for example, a broker can trigger an email or SMS to the merchant with the offer details populated (using variables like `{all_offers_all_details}` which formats a list of offers) ²⁷ ¹²⁰ . This can be used to send a quick **"You're approved!" email or text** with key terms, directly from the system. Brokers can also add notes or internal status to each offer, like "merchant requested better terms" or "offer revised". MCA Pilot's pipeline Kanban (Offers Pipeline) lets the team drag or update offers through stages: **New** (just in), **Pitched** (communicated to merchant), **Contracts Requested** (merchant wants contract), **Contracts Out** (contract sent to merchant), and possibly **On Hold** or **Dead** for offers that didn't move ¹²¹ . In the Offers Pipeline screenshot, we see columns New, Pitched, Contracts Requested, Contracts Out, with counts for each – users can move an offer card from one stage to the next as the deal progresses ¹²¹ . For

example, when the rep speaks to the merchant and pitches the offer, they'd mark it as Pitched. If the merchant agrees and asks to move forward, the rep marks it Contracts Requested, sends out contracts (the Request Contracts feature likely helps here, possibly by sending a DocuSign link), and then marks it Contracts Out ¹²¹. If the contract comes back signed and funding is arranged, the rep will mark the offer as **Funded**. In MCA Pilot, this last step is formalized: the user clicks a **"Mark as Funded"** button on the offer which opens a form to input funding details ¹²² ¹²³.

When marking an offer funded, the broker enters specifics like: actual funded date, the **commission amount** earned, any **broker fee** charged, and expected dates for those payouts ¹²⁴ ¹²⁵. They can also select how **payments will be split** if needed (e.g. if two reps split commission, or if it's a syndicated deal). After confirming, the system changes that offer's status to Funded (blue label) and creates an **Advance record** for the deal ¹²⁶ ⁹. The deal is now considered completed/funded. The offer leaves the active Offers Pipeline and appears in the **Advances** section (for tracking post-funding performance). If an offer is not taken by the merchant (they declined it), the rep might mark it as **Lost** or simply not mark it funded; such offers might remain in an "inactive" state or could be archived after some time.

Automation: Some aspects of offer management are automated by the earlier modules – e.g., an offer record is auto-created by AI parsing. The pipeline movement is mostly manual (brokers update stages), but MCA Pilot likely automates certain reminders. For instance, any offer in "Contracts Out" for too long might trigger a follow-up task or email to the merchant. The system's **Follow-Ups** feature can automate touchpoints based on offer status: e.g., send a text to merchants with approved offers reminding them of the terms every X days ²⁷ ¹²⁷, or if an offer was declined by funders, automatically email the merchant on the 1st of next month asking for new statements to try again ²⁷ ¹²⁷. This is configurable by status. Another automation: once an offer is marked funded, the system calculates the **Paid-in Percentage** over time. The Advances module knows the term and payment frequency, so each day or week it can estimate how much of the advance is paid back and update the percentage ¹¹. If the platform integrates with payment data or the funder's portal, it could update actual payments – but even without that, it can assume a straight-line payment and compute approximate % paid. This percentage is used to flag when a deal reaches, say, 50% paid (common point to offer a renewal). MCA Pilot presumably highlights this in the Advances list and possibly triggers a **Renewal Reminder** automation or at least a report of "Deals up for Renewal". We saw that they tout calculating paid-in so you call at the right time ¹¹.

Dependencies: Offer management sits between submissions and advances. It relies on **submission outcomes** to create offers. It also ties into the **communications** module for sending offers to merchants (via email/SMS templates). The final step of marking funded relies on the **Commissions management** setup – user inputs expected commission, but after funding, they need to track if that commission was received, which happens in the Advances/Payments module. So an Offer marked funded creates an Advance with initial expected commission entries. The Offer Pipeline also depends on having the concept of **stages** distinct from submission status: it's more about the merchant's decision stage, whereas submission status was about lender's decision. MCA Pilot separates these (notice an offer has its own status/stage like pitched/contracts out, while the submission that spawned it might have status approved) ³. This requires the system to manage two parallel but linked statuses.

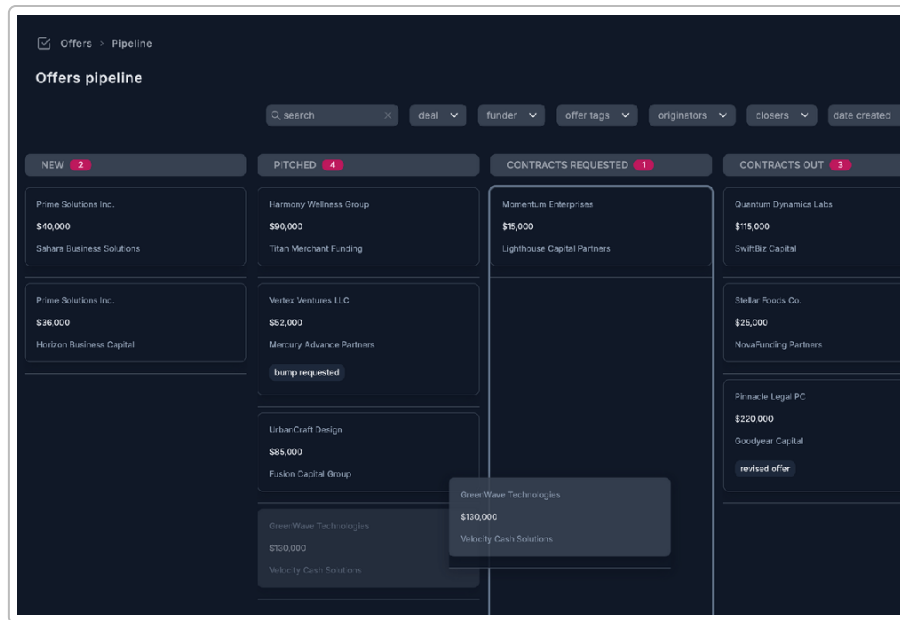
Recreating Guidance: To implement offer management, start by allowing each submission to optionally result in one or more **Offer records**. Model an Offer with fields: `funding_amount`, `factor_rate`, `term`, `payment_frequency`, `commission_rate` or `points`, any stipulations text, and a link to the originating submission/funder. Also include a field for **offer_stage** (with values like New, Pitched, Contracts Out,

Funded, etc.) and perhaps **offer_status** (active vs lost vs funded). When a user manually logs an approval (like a “Mark as Approved” action on a submission), have a form for them to input the terms ¹²⁸ ¹¹⁹ and then create the offer record ¹²⁹. Display offers on the deal page (maybe a tab or section listing all offers with their statuses). Create a global **Offers Pipeline view** that queries all offers where status is active (not funded or lost) and groups them by stage. This can be a drag-and-drop Kanban or simply click to change stage. To encourage usage, design it such that when an offer is moved to Funded stage, it prompts the funding details form.

For marking funded, create a **Funding form** that captures: `funded_date`, `gross_funding_amount` (could be same as offer amount or adjusted if partial), `broker_commission_amount` (or percentage to calculate amount), `broker_fee`, and any splits or notes ¹²³ ¹³⁰. Submitting this form should create an **Advance** entry containing those details. Also, update the offer’s status to funded and maybe the deal’s overall status to funded. In the Advance record (detailed next module), you’ll track payout and repayment.

Implement some **automated triggers**: e.g., if an offer is created and not moved for a few days, remind the rep to pitch it. Or integrate the Follow-up scheduler: allow user to set up rules like “send this SMS template to all deals in ‘Approved’ status daily” ²⁷ – in your case, tie it to offers perhaps. At minimum, you can introduce a scheduled task that lists all active offers so the team can review regularly. The renewal logic can be part of Advances, but ensure that when an advance hits X% paid, an alert or list is generated for sales to call those clients for renewals.

Example: A deal has two offers from two funders. In the Offers Pipeline, Card 1 is “Prime Solutions Inc – \$40,000 offer from **Sahara Business Solutions** (lender)” under *New*, and Card 2 is “Vertex Ventures LLC – \$52,000 offer from **Mercury Advance**” under *Pitched* with a tag “bump requested” (meaning merchant asked for a better offer) ¹²¹. The rep negotiates a bit, Mercury comes back with a revised offer, the rep updates that offer’s terms or marks it “revised offer” (as seen for Pinnacle Legal PC’s card) ¹²¹. The merchant decides to go with Mercury’s \$52k. The rep moves that card to Contracts Requested and then to Contracts Out after sending the contract. Once signed, the rep clicks *Funded*, fills in that they expect \$5,000 commission to be paid on a certain date, and confirms. The system now logs \$52k funded with Mercury, moves the deal to funded status, and that offer disappears from the pipeline (or is marked completed). The rep and management can now track this funding in the Advances list, and the other offer from Sahara might either expire or remain as an unused approval (could be marked lost or left as is, depending on policy).



Example of the Offers Pipeline view, where approved offers are tracked by stage. Each card is a lender's offer for a deal, moving from New to Pitched to Contracts Out until Funded (or dropped). Note tags like "bump requested" or "revised offer" indicating negotiation on terms ³.

Post-Funding Management (Advances, Commissions & Renewals)

Purpose: Funding a deal isn't the end of the relationship – the brokerage will track the funded deal for commission payments, monitor the merchant's repayment progress (for renewals or potential defaults), and possibly handle any **reverse consolidations** or participations. MCA Pilot's Advances module is essentially a ledger of all funded deals (advances) and their ongoing status ¹³¹ ¹³². It allows the team to see how many deals have been funded, the amounts, which funder funded it, how much has been repaid (% paid in), and whether the deal is "on track" or has issues (missed payments or defaults) ¹³³. It also helps the back-office ensure that expected **commission and fee payments** from funders have been received and allocated to the right reps. And for complex cases like **reverse consolidations** (where the new advance pays off existing advances from other funders), the system can schedule the distribution of payouts to those prior funders over time.

User Workflow: When an offer is marked funded, MCA Pilot creates an **Advance record**. Users can access all advances via an **"Advances" page in the sidebar** ¹³⁴ ¹³⁵. There, they see a table with each funded deal's details: merchant (deal) name, the funder who funded it, the funded amount, the current status of that advance, the paid-in percentage, the originator/closer responsible (for credit and commission), and the date funded ¹³¹ ¹³². For example, "Stellar Foods Co. – NovaFunding Partners – \$25,000 – on track – 11.37% – Victor (originator) – funded 13 days ago" ¹³³. The status "on track" likely means payments are coming in as expected; other statuses might be "missed payments" or "defaulted" as seen in the list ¹³³, which the user can set if they get word of issues (or possibly integrate via data from funders). If a deal goes into default or misses a certain number of payments, the broker can update its status accordingly (Pinnacle Legal PC shows "missed payments" status at 74% paid, Momentum Enterprises "defaulted" at 39.67% paid)

¹³³.

Within each advance's detail, there's typically a **Payments or Commission tab**. MCA Pilot allows users to manage expected and actual **commission payments** from funders. When marking funded, expected commission and fee are logged with expected dates ¹²⁵ ¹³⁶. The system can automatically **schedule those payouts** – clicking an “Auto add payouts” button will generate records for the commission and any broker fee due, and mark them as expected on the given dates ¹³⁷ ¹³⁸. Later, when the brokerage actually receives the commission from the lender (say via ACH a week later), a user can mark that commission as **paid** by recording a payment entry (or the system might detect it via QuickBooks integration, etc., but likely manual). The **Add Payment** function lets the user log a payment against the advance: they select type “commission” (or “fee”), enter the amount received and the date ¹³⁸ ¹³⁹. This moves the commission from expected to collected, helping the company track its revenue. If multiple reps split commission, the “Payment splits” setting would allocate portions automatically when adding a payment ¹⁴⁰ (for example, 50/50 between an originator and closer if defined).

For **Reverse Consolidation** scenarios, when marking an offer funded, the user might indicate that, say, \$30k of the \$100k is used to pay off 2 other advances the merchant had. MCA Pilot can schedule **weekly distribution payments** to those prior funders from the new advance's daily payments ¹⁴¹ ¹⁴². The platform likely has a way to record the details of the balances being paid off and then automatically generate “payout” entries (maybe under a Distributions tab) each week for the old funders. A video is referenced for how to set this up ¹⁴². Essentially, the system would subtract those payoff amounts from what the merchant pays and remind the broker to pay the other funders weekly for their share. This is a niche but important feature for advanced use cases, ensuring the broker doesn't forget to forward due amounts to other funders in a consolidation.

Finally, as advances progress, MCA Pilot's dashboard or reports might highlight deals that are e.g. **50% paid** (common renewal point) or 100% complete. Reps can use a **Renewals filter** to see which merchants are likely ready for another loan. The system can also automate renewal outreach: e.g., for deals at 50% paid and status “on track,” trigger an email or task to contact them about renewal (this could be configured in Follow-Ups as “Status = on track AND paid_in >=50% → send renewal offer template”).

Automation: Key automations in post-funding include the **Paid-In calculation** – MCA Pilot automatically calculates the percentage paid by comparing the days since funding vs. term, or if integrated, actual payments made ¹¹. For example, if a deal is 6 months term and we're 3 months in, it might show ~50% paid (assuming equal payments). If “missed payments” status is set, maybe the system slows or stops increasing the paid-in percentage, highlighting an issue. The platform may also auto-flag if an advance goes beyond expected end date without being fully paid (implying some default). **Auto Add Payouts** is another automation: with one click, all expected commission and fee payments are generated, and if splits are defined (like two reps splitting a commission), those distribution entries are created too ¹³⁷ ¹⁴³. This saves manual entry of separate commission lines.

If a **rep's compensation** is tracked, the system might also auto-calculate their share of commission based on a predefined split or commission percentage. For instance, if a junior broker gets 50% of commission and the house gets 50%, the system can log two payout entries (one to broker, one to house) from the single commission amount.

Another automation is using the **Advances data for analytics**: the dashboard might roll up total funded volume, average commission, default rates, etc., all driven by the advances records.

Dependencies: The Advances module depends on the **Offer Funding process** to feed it initial data. It also ties into any **accounting integrations** (if present, to sync payments or revenue). Team permissions come into play: you might restrict that only managers see the dollar amounts in commissions and payments (MCA Pilot has a setting to restrict reps from seeing total company payments/commissions ¹⁴⁴). The Renewal process depends on Advances data, as described. And if the brokerage uses external tools for payment processing, those could be integrated via webhooks or manual import to update paid-in or status (though not explicitly described, it's possible to extend via the Custom Workflows webhooks ¹⁴⁵).

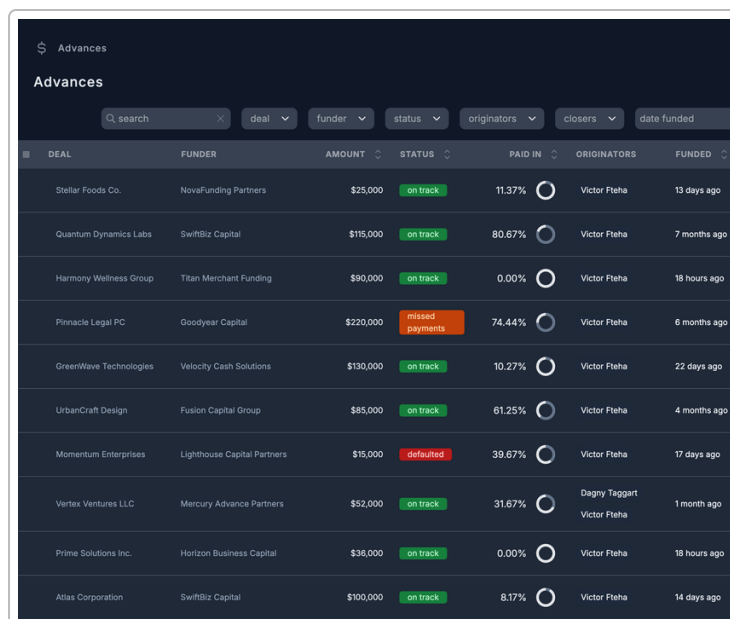
Recreating Guidance: To create a post-funding tracking module, define an **Advance entity** that links to Deal, has fields for funder, funded_amount, funded_date, term_length, payment_frequency, and statuses like on_track, delinquent, etc. Also include fields for total_expected_payments (which might equal funded_amount * factor if factor rate) and maybe a schedule for payments. For simplicity, you can calculate paid_in as (current_date - funded_date) / term_length (in days) * 100% for a rough estimate, but ideally track number of payments made vs. total expected. If daily payments, and term 120 days, each day paid increments ~0.83%. This calculation can run as a daily job updating all advances. Provide a UI listing all advances with columns as seen (which is straightforward from the Advance records plus some computed fields).

For commissions, have a related **Payout** or **Payment** entity tied to an advance. When marking an offer funded, create payout entries for "Commission expected" and "Broker fee expected" if applicable. Each payout entry should have amount, type (commission/fee), expected_date, received_date, and optionally assigned_to (who gets this money, e.g. rep or company). The "Auto-add payouts" can simply take the data input at funding time (e.g. commission \$X expected on Y date) and generate those records automatically ¹³⁷ ¹³⁸. Later, when a payment is received, allow the user to mark it received by entering the actual date (could double as a simple accounting ledger – Phase 2 you might integrate with QuickBooks or similar to avoid double entry). Also incorporate **split logic**: you might have a configuration where each deal or each rep has a commission split. For MVP, you can skip split and assume one commission per advance. Later, add ability to split: maybe store an array of recipients and percentages for each advance. The UI to manage that could be on the deal assignment (e.g. originator gets X%, closer Y%). Then, when adding a payment and selecting splits, auto-create two records (one for each person's share).

Implement a way to mark an advance's **performance status**: on track, behind, default. Could be manual (a dropdown on advance record). If a user marks "missed payment", maybe prompt for notes or how many. You could also incorporate logic: if today > expected end date and paid_in < 100%, auto mark default or prompt user. For reverse consolidation payouts, you would create additional records: essentially, if an advance includes paying off others, you might link those advances in a consolidation group. Then schedule outgoing payments to those other funders. This could be complex to fully automate; an easier way is to allow the user to input something like "\$X of this advance is paying off Funder Y, to be paid in N weekly installments." Then generate a series of payment reminders or entries (type "distribution") for those. This ensures you have a record to check off each week when you send money to prior funder.

Example: On the Advances page, a manager sees: Quantum Dynamics Labs – funded \$115,000 by SwiftBiz Capital 7 months ago, status "on track", 80.67% paid in ¹³³. They know it's nearing payoff and might call for renewal. Another shows Pinnacle Legal PC – \$220,000 by Goodyear Capital 6 months ago, status "missed payments", 74.44% paid ¹³³. They'll need to follow up on those missed payments or note the default risk. Each of those has entries in their detail showing if the commission was received. The manager can filter this view by originator to see each rep's funded volume and commissions. In terms of building one step at a

time: Phase 1 you might not implement the entire Advances module; you could consider a deal “Funded” and maybe export data manually. But given the question context (“full-scale competitor”), you would plan to include this in Phase 2 after core CRM and submission features, since tracking funded deals and commissions is crucial for the business.



DEAL	FUNDER	AMOUNT	STATUS	PAID IN	ORIGINATORS	FUNDED
Stellar Foods Co.	NovaFunding Partners	\$25,000	on track	11.37%	Victor Fleha	13 days ago
Quantum Dynamics Labs	SwiftBiz Capital	\$115,000	on track	80.67%	Victor Fleha	7 months ago
Harmony Wellness Group	Titan Merchant Funding	\$90,000	on track	0.00%	Victor Fleha	18 hours ago
Pinnacle Legal PC	Goodyear Capital	\$220,000	missed payments	74.44%	Victor Fleha	6 months ago
GreenWave Technologies	Velocity Cash Solutions	\$130,000	on track	10.27%	Victor Fleha	22 days ago
UrbanCraft Design	Fusion Capital Group	\$85,000	on track	61.25%	Victor Fleha	4 months ago
Momentum Enterprises	Lighthouse Capital Partners	\$15,000	defaulted	39.67%	Victor Fleha	17 days ago
Vertex Ventures LLC	Mercury Advance Partners	\$52,000	on track	31.67%	Dagny Taggart Victor Fleha	1 month ago
Prime Solutions Inc.	Horizon Business Capital	\$36,000	on track	0.00%	Victor Fleha	18 hours ago
Atlas Corporation	SwiftBiz Capital	\$100,000	on track	8.17%	Victor Fleha	14 days ago

Example of the Advances (funded deals) dashboard: It lists each funded deal, the funder, amount, status (on track, missed payments, defaulted), paid-in %, assigned reps, and when it was funded. This helps monitor portfolio performance and commission collection ¹³¹ ¹³² .

Team Management, Collaboration & Permissions

Purpose: MCA Pilot is built for brokerage teams, so it includes features for managing multiple users, controlling data access, and collaborating on deals. The goal is to let each role (rep, manager, admin) work with the system in a tailored way – e.g., sales reps only see their own deals, managers see all or their team’s deals, and certain financial info (like total commissions) can be restricted to admins ¹⁴⁶ ¹⁴⁷ . Collaboration features like internal notes and notifications ensure everyone stays on the same page about each deal’s progress without resorting to external spreadsheets or endless email threads.

User Workflow: User Accounts & Roles: An admin can invite team members into the system and assign roles/permissions. MCA Pilot likely has at least an **Admin/Super Admin** role and a **Standard User (Rep)** role, possibly with finer granularity. Super Admins can see and do everything (all deals, settings). A normal rep might be limited to only deals where they are an **Originator or Closer** (those assigned to them) ¹⁴⁶ . The system “role-based permissions” feature allows, for example, a junior ISO to see only their deals, while a manager could see deals of everyone under their management hierarchy ¹⁴⁶ . This is configured in user settings or an admin panel. The prompt from MCA Pilot says “We’ll set up role-based permissions so that reps can only see their own deals, while managers can see anyone on their team’s” ¹⁴⁶ , implying a hierarchy configuration (perhaps teams or manager field on user profiles).

Deal Assignment: When creating or editing a deal, the admin or rep can assign one or more **Originators** and **Closers** to that deal ⁵ ⁷ . Originator typically means the person who brought in the lead, Closer might be a person assisting or a team lead. The **Primary Originator** can be marked for leaderboard/credit purposes ⁷ ¹⁴⁸ . This assignment not only determines visibility (a rep might only see deals where they're assigned) but also can feed into performance metrics (leaderboards, conversion rates per rep, etc.) and commission splits.

Collaboration: Inside each deal, users can leave internal **Comments/Notes**. MCA Pilot provides a **Notes tab or activity feed** with rich-text commenting and file attachments ¹⁴⁹ . Comments are time-stamped and likely label the author, allowing a team to discuss the deal internally (e.g., a manager writes "Let's push for a higher offer, merchant has another option" or a rep writes an update after calling the client). If a user mentions or tags a teammate in a comment, the system triggers a **Notification** to that person ¹⁵⁰ ¹⁵¹ . MCA Pilot has a **Notification Center** that collects these alerts so users can see what's new (e.g., "@John commented on Deal X") ¹⁵⁰ . This reduces the need for external messaging – the team can coordinate within the app.

Productivity Aids: The interface includes things like **Unified Search (Cmd+K)** to jump to any deal or function quickly ¹⁵² , and a **Dark/Light Mode** toggle for user preference ¹⁵³ . While not directly functional modules, these design touches improve user experience and efficiency. There's also a mention of **Collapsible Sidebar** and mobile-responsive (PWA) design, meaning the team can use the tool on the go on any device ¹⁵⁴ ¹⁵⁵ .

Lead Management: Although MCA Pilot is deal-centric, it does track **Lead sources and marketing analytics**. There's a field for **Lead Source/Lead Type** on deals (e.g., channel or campaign) ¹⁵⁶ ¹⁵⁷ . This allows managers to see which sources produce funded deals and the ROI per source ¹⁵⁸ . The system can generate reports or at least aggregate totals by source (maybe in the dashboard). It helps in decision-making on marketing spends. Also, **batch tracking** might be present (the docs mentioned "Lead Sources and Batches" in related articles ¹⁵⁹ – likely the ability to tag a batch of leads for a specific campaign and track performance).

Permissions on Financial Data: The docs hint at a setting: "How do I restrict my reps from seeing total payments/commissions to the company?" ¹⁴⁴ . This suggests you can hide the overall commission or payout info from standard users. For example, a rep might see their commission on a deal but not the total that the ISO company made (if the company added extra points or fees). This kind of permission would be configurable in roles.

Automation: Team management itself is not heavily automated, but notifications and tagging are. For instance, anytime a deal is assigned to a rep, they might get an alert or email. If an offer is marked funded, perhaps the system credits the primary originator on a leaderboard automatically. The **Leaderboard** likely calculates each user's funded volume, number of deals, total commissions, etc., which could be an automated report (the mention in docs about "Primary Originator: user who gets credit for the deal on the leaderboard" ⁷ suggests an automated tally). Also, the system's data presets mean new accounts come with pre-configured pipelines and templates, which might include a default user role setup and sample workflows for teams ¹⁶⁰ ¹⁶¹ .

Dependencies: This module ties into everything because permissions gates access to deals, submissions, etc. If a rep shouldn't see others' deals, the queries for deals, submissions, offers, advances all need to filter

by ownership. Comments and notifications depend on an internal messaging system (Intercom is referenced for support, but in-app notifications are custom for the platform). Also, integration with external email is possible for notifications (e.g., send an email to a user when they're tagged or assigned something), but many prefer in-app or Slack integrations (not sure if MCA Pilot has direct Slack integration, but webhooks allow connecting to such if needed). The team management also interacts with commission splits on advances (if multiple users on a deal, the splits may be determined by those roles).

Recreating Guidance: For user management, leverage a robust auth system with role support. Define roles like Admin, Manager, Rep. Implement permission checks such that: Reps can only query deals where they are in the deal's assigned users. Managers might have a field linking them to their team's users and thus can see deals of those users. If a simple approach, you could have "owner" field for deal and only allow that. A more complex but flexible way is to have an Access Control List linking users to deals with permission type. However, given the typical structure: one or multiple originators/closers per deal, you can use that as the access list. On the frontend, filter lists and search results by those permissions.

Add a **Notes/Comments system**: at simplest, a notes table with deal_id, user_id, content, timestamp. For rich text and tagging, you might integrate a rich text editor that can handle "@" mentions. When saving a comment, parse for "@Name" to find if it matches a user, and if so, create a notification entry like (to_user, from_user, deal_id, comment_id). Provide a **Notifications UI** – could be a bell icon showing unread notifications. Each notification links to the relevant deal or comment. Optionally, send an email for important mentions. Use websockets or polling to update notifications in real-time if possible.

Ensure every major update (new comment, status change, task assignment) can produce a notification. For assignments: if an admin assigns a rep to a new deal, generate a notification "You have been assigned to Deal X." For follow-up tasks: if AI creates tasks for stips, maybe assign those tasks to the deal's owner and notify them. The system should help keep everyone informed without direct communication.

Implement the **Leaderboard** or basic reporting for team performance: either on the dashboard or a dedicated page, show metrics per user (count of funded deals, total funded amount, total commission). This uses data from advances and assignments. That can be Phase 2 as a nice-to-have analytic.

Phase Prioritization: Team management features can be phased – Phase 1 might just have a single admin user scenario (or minimal permissions). Phase 2, once multiple users are on-boarded, implement full role restrictions and notifications. Collaboration (comments) is extremely helpful but could also come in Phase 2 after core deal functions. However, given it doesn't require external integration, it's often easy to include early for user convenience.

Integration & Custom Workflow Automation

Purpose: MCA Pilot recognizes that broker shops use various tools (from e-signature services to marketing platforms), so it provides integration points to avoid re-entering data. The platform offers **Custom Webhooks and API** for extensibility ¹⁴⁵, and the team behind it even promises quick custom integrations built on request ¹⁶² ¹⁶³. The idea is that if something isn't handled natively (say, a specific dialer or a lead source), you can connect it via Zapier/Make or direct webhooks so that MCA Pilot becomes the central hub of truth for deals.

Capabilities: They list “Custom notifications and webhooks” and “Custom integration with your application form” as part of the package ¹⁶⁴ ¹⁶⁵ . For example, a broker might have a lead form on their website or use a landing page builder – instead of manually creating a deal from each submission, they can set up a webhook from that form to MCA Pilot’s API to create the deal automatically (mapping form fields to deal fields). The documentation mentions integrations with **DocuSign, PandaDoc, Dropbox Sign, JotForm, etc.** where the MCA Pilot team will map all the fields to the CRM ³⁴ . So if a merchant fills out and signs a DocuSign application, that data goes straight into MCA Pilot (likely through an integration using those platforms’ APIs or webhooks). Similarly, they have pre-built API integrations for certain funders (like Credibly, Rapid Finance, etc. – these are likely direct submission APIs bypassing email) ³⁷ ¹⁶⁶ .

User Workflow: If a user wants to integrate, say, JotForm for lead capture, they’d follow an MCA Pilot guide to set up a webhook from JotForm submissions to MCA Pilot’s endpoint (the docs mention “Integrate a new rep’s JotForm” ¹⁶⁷). Usually, this involves retrieving an API key from MCA Pilot (the user can create one in Settings > API Keys) ¹⁶⁸ ¹⁶⁹ , then configuring the external app’s webhook URL with that key. Once done, any new form submission triggers the creation of a deal (with owners assigned if logic is set to, e.g., round-robin assign to reps, etc.). Users likely test it and then from that point, their external forms feed directly into the MCA Pilot pipeline, ensuring no leads are dropped.

Webhooks in the other direction allow MCA Pilot to notify external systems of events. For instance, when a deal is funded, you could have a webhook to a Slack channel or to your accounting system. Or when a new deal is created, a webhook could send the data to a dialer or marketing automation. The presence of webhooks means an admin user can set a URL for certain triggers in the system. If using Zapier/Make, those services can catch the webhook and then do various tasks. This flexibility means even if MCA Pilot doesn’t natively have a feature (like a power dialer or a drip email sequence for unresponsive leads), you can connect one fairly easily.

Automation: Through these integrations, a lot can be automated: *Lead attribution* was mentioned ¹⁵⁶ – that likely involves capturing UTM parameters or source info from the web form into MCA Pilot. With webhooks, you can have each ad campaign tagged and the system will roll up ROI. *Email marketing* could be connected – e.g., new deals could be pushed to Mailchimp lists or vice versa. There’s mention that MCA Pilot doesn’t do full CRM marketing (NextLevel claims MCA Pilot lacks CRM beyond deal tracking ¹⁷⁰ ¹⁷¹), but with integrations you could sync contact data out to a marketing tool if needed.

Another integration point is **Phone/SMS**. MCA Pilot likely integrates with services for phone calls or texting (or at least supports click-to-call via a softphone). If not built-in, a webhook or API could send an SMS via Twilio when certain events happen (though NextLevel includes SMS, MCA Pilot may require integration). The platform’s reliance on PWA for mobile means push notifications might be supported as well for immediate alerts.

Dependencies: To enable these, you need stable **APIs** on the platform side. MCA Pilot must expose endpoints for creating deals, reading/writing data, etc. It likely has an API or uses something like n8n (the presence of `n8n.mcapilot.com` in links for forms suggests they utilize n8n workflows for some integrations ¹⁷² ¹⁷³). If building a competitor, designing a clean REST API early is important for these extensibility features. Security (API keys, scopes) is also vital since sensitive financial data is in play.

Recreating Guidance: For Phase 1, focus on core internal features. But plan the architecture such that you can add an **API layer**. This layer should allow creating and updating deals, adding documents (perhaps via

URL or base64 upload), retrieving data, etc. Implement an **API key management** in the app (as MCA Pilot has Settings > API Keys ¹⁷⁴). Make endpoints for important objects (deals, funders, submissions, offers, etc.) so that external systems can at least create and maybe query them. Simultaneously, allow **outgoing webhooks**: decide on which events (deal created, status changed, offer created, etc.) you'll support. For each, let the user configure a webhook URL and maybe a filter (or you could do one global webhook that sends everything with event type, and external side filters it). Many modern apps also integrate with Zapier by offering a Zapier app – but providing a generic webhook is a simpler universal approach.

To integrate application forms, if you have the API ready, a low-code tool or Zapier can be used. You might eventually create a small helper for popular form builders: e.g., a JotForm webhook URL that accepts their form fields and maps to your API. MCA Pilot likely built custom n8n workflows for each major form provider which they deploy for their customers (n8n is an automation tool like Zapier that can be self-hosted; the `n8n.mcapilot.com` links hint that they use it to host the upload forms and scanning forms ¹⁷² ¹⁷³). In building a competitor, you could either directly integrate (hard-code field mapping from known forms) or provide a mapping UI for custom fields – but that might be overkill initially. Instead, do what they did: set up a generalized form intake endpoint where a JSON of field names can be posted to create a deal, then provide guidance for connecting external forms to it.

Also, incorporate a **webhook for funder responses** as an alternative to IMAP parsing in the future. Some funders might be able to send HTTP callbacks on their decisions (though email is the standard, a few might offer API callbacks). If so, supporting that would skip the email parsing step for those funders and get the data more directly.

Phase Prioritization: Integration capabilities are typically Phase 3 once the core system is robust. However, at least a basic API and form intake might be needed by Phase 2 to not slow down operations (since manual data entry of leads is a pain). So prioritize the most impactful ones: integrating the application form (so deals flow in automatically) early on, and adding other integrations (funders API, webhooks for Slack/Email alerts) as you go.

Development Phases & Priorities

Building an MCA platform from scratch is a significant project; breaking it into phases helps ensure core value is delivered early while planning for advanced features later. Below is a suggested phase-wise breakdown:

- **Phase 1: MVP – Deal Tracking & Basic Submissions**

Focus on the essentials that let a small team manage deals and fundings. Implement the **Deal Pipeline** with ability to create deals, upload documents, and manually track statuses. Include basic **Funder Management** (at least store lender contacts and send emails) and **One-click Submission Emails** without AI parsing initially. Users should be able to add funders to a deal and click send to generate pre-filled emails ⁷³. Basic **Offer logging** can be manual: allow users to mark submissions as approved/declined and record offer terms ¹¹⁸ ¹¹⁹. Also, implement **User management** for at least admin vs rep roles so data can be restricted ¹⁴⁶. At this stage, many advanced automations (AI parsing, auto-matching, etc.) can be done manually by users through the UI. The goal is to replace spreadsheets and email by having one system to log deals, store files, submit to lenders, and record

outcomes. Ensure document upload, tagging/assigning deals to reps, and a basic **Advances list** to mark deals funded and record commission expected. MVP success is when brokers can go from new lead to funded deal entirely within the system (albeit with some manual data entry).

- **Phase 2: Full CRM & Workflow Automation**

Now add the productivity and CRM features around the core. This includes **Auto Funder Matching** by implementing the criteria filters for funders vs. deals ³⁵ ³⁸, so reps get an auto-generated list of qualified lenders instead of picking blindly. Introduce **AI-assisted data entry**: e.g., **Application Form Integration/Intake** (webhooks or direct integration to create deals from online forms) and possibly an **AI App Scan** to populate deals from PDF apps ³⁶ ³⁷. Also bring in **Follow-Up Automation** – allow creating email/SMS templates tied to deal statuses (so the system can automatically chase missing documents or send renewal reminders) ²⁹ ¹²⁷. The internal CRM capabilities should be expanded: add a **Dashboard** with KPIs (deals in each stage, total funded, conversion rates) ¹⁷⁵ ¹⁷⁶, a basic **Leaderboard** for reps, and perhaps simple marketing tracking (lead source attribution reports ¹⁵⁶). In this phase, also refine **Team features**: comment tagging notifications, role-based access fine-tuning, and make the UI slick (search bar, light/dark mode toggle for user preference ¹⁵² ¹⁵³). By the end of Phase 2, the platform should feel like a comprehensive CRM: handling not just the transaction with lenders, but also internal sales workflows and external follow-ups. It should save significant time through features like automatic lender filtering and scheduled emails.

- **Phase 3: Advanced AI & Integrations**

This is where the system moves toward the “autopilot” vision. Implement **AI Email Processing** fully, so lender responses are read and parsed to auto-update submissions and create offers ¹² ¹³. Roll out **AI Bank Statement Underwriting**, which extracts financial metrics and possibly detects existing positions from uploaded statements ³². With that, you can also implement more sophisticated **deal scoring** or flagging (e.g., highlight if the deal likely fits certain top-tier lenders or if it has red flags). Expand the **Funder Criteria AI** to quickly onboard new lender programs by scanning PDFs ⁴² ⁴⁰. This phase also includes deep integrations: build out the **Direct API connections** to any major funders who have them (so submissions to those funders happen via API, and even their status updates could feed back via API if available). Add integrations for **phone dialers, SMS gateways, and calendar/task systems** if needed – for example, click-to-call from within a deal, or syncing follow-up tasks to Google Calendar or a task app. At this stage, **virtually every repetitive task is automated**: new leads flow in automatically, documents get OCR’d and organized, best-match lenders are suggested, emails go out with one click, replies update the system, and even post-funding, the system reminds you about commission receipts and renewal opportunities. Users mainly supervise and handle the human interactions (with merchants and any negotiation). Phase 3 may also introduce more **AI-driven insights**, like predicting which deals are likely to fund (using historical data), or an AI chatbot for initial lead qualification (beyond scope, but conceivable). Essentially, Phase 3 solidifies the platform as a cutting-edge tool that can do much of the heavy lifting – delivering on the “MCA Pilot” name by having the system pilot many processes automatically ⁴⁶.

Throughout all phases, gather user feedback (brokers will have lots of practical insights) and iterate. Ensure each phase’s additions integrate smoothly with previous ones – e.g., when adding AI, make it override or fill the same fields users used to fill manually, so it’s consistent. Also, maintain flexibility: not every shop might

want every feature (some may trust their gut over the AI, especially initially), so allow them to toggle certain automations on/off.

By following this phased approach, you'd build a robust competitor: starting with a solid foundation of deal tracking (what NextLevel described as "they do deals" – which is crucial) ¹⁷⁰, then layering on full CRM capabilities (so you provide "everything else" on top of deal tracking), and finally adding the advanced AI that truly differentiates and saves time ⁴⁶. This way, even if the AI falls short occasionally, the system is still extremely useful; and as the AI improves, it gradually takes over more work, which is exactly the trajectory MCA Pilot has pursued.

¹ ³ ⁸ ¹¹ ⁷³ ⁷⁴ ⁷⁵ ⁷⁶ **MCA Pilot: MCA on Autopilot**
<https://mcapilot.com/>

² ²⁵ ²⁶ ³¹ ³² **I manually uploaded bank statements to a deal. How do I get the AI to underwrite the financials/positions of the deal, and to check the files to see if something is missing? | MCA Pilot Help Center**
<https://docs.mcapilot.com/en/articles/12312357-i-manually-uploaded-bank-statements-to-a-deal-how-do-i-get-the-ai-to-underwrite-the-financials-positions-of-the-deal-and-to-check-the-files-to-see-if-something-is-missing>

⁴ ⁵ ⁶ ⁷ ¹⁴⁸ **How to create a Deal | MCA Pilot Help Center**
<https://docs.mcapilot.com/en/articles/10799377-how-to-create-a-deal>

⁹ ¹¹⁷ ¹²² ¹²³ ¹²⁴ ¹²⁵ ¹²⁶ ¹³⁰ ¹³⁶ **How to Mark an Offer as Funded in Fundmore (Create an Advance) | MCA Pilot Help Center**
<https://docs.mcapilot.com/en/articles/10895293-how-to-mark-an-offer-as-funded-in-fundmore-create-an-advance>

¹⁰ ⁸¹ ⁸² ⁸⁶ ⁸⁷ ¹¹² ¹¹⁸ ¹¹⁹ ¹²⁸ ¹²⁹ **How to mark a Submission as approved | MCA Pilot Help Center**
<https://docs.mcapilot.com/en/articles/10892279-how-to-mark-a-submission-as-approved>

¹² ¹³ ⁴⁶ ⁴⁷ ¹⁰⁰ ¹⁰¹ ¹⁰² ¹⁰³ ¹⁰⁴ ¹⁰⁵ ¹⁰⁶ ¹⁰⁷ ¹⁰⁸ ¹⁰⁹ ¹¹⁰ ¹¹¹ ¹¹³ ¹¹⁴ ¹¹⁵ **AI Processing | MCA Pilot Help Center**
<https://docs.mcapilot.com/en/articles/10058101-ai-processing>

¹⁴ ¹⁵ ⁶⁹ ⁷⁰ ¹⁵⁴ ¹⁵⁵ ¹⁶⁴ ¹⁶⁵ **MCA Pilot | Pricing**
<https://mcapilot.com/pricing>

¹⁶ ¹⁷ ²³ ²⁴ ⁵⁰ ⁹⁸ ⁹⁹ ¹⁶⁰ ¹⁶¹ ¹⁶² ¹⁶³ **Features - NextLevel MCA | 50+ MCA Tools in One Platform | NextLevel MCA**
<https://nextlevelmca.com/features>

¹⁸ ¹⁹ ²⁰ ²¹ ³³ **AI Rename: Automatically Standardize Bank Statement File Names | MCA Pilot Help Center**
<https://docs.mcapilot.com/en/articles/13188748-ai-rename-automatically-standardize-bank-statement-file-names>

²² ⁵⁹ ⁷⁷ ⁷⁸ ⁷⁹ ⁸⁰ ⁸³ ⁸⁴ ⁸⁸ ⁹² ⁹³ ⁹⁵ ⁹⁶ ⁹⁷ ¹¹⁶ **Connect Your Email to MCA Pilot | MCA Pilot Help Center**
<https://docs.mcapilot.com/en/articles/9268937-connect-your-email-to-mca-pilot>

²⁷ ²⁹ ¹²⁷ **Follow Ups | MCA Pilot Help Center**
<https://docs.mcapilot.com/en/articles/12341841-follow-ups>

28 30 120 173 Variables for Follow Ups | MCA Pilot Help Center

<https://docs.mcapilot.com/en/articles/11871178-variables-for-follow-ups>

34 64 65 89 90 91 145 146 147 149 150 151 152 153 156 157 158 175 176 MCA Pilot | Features

<https://mcapilot.com/features>

35 38 39 51 54 55 56 57 61 62 66 How to match funders automatically for a deal? | MCA Pilot Help Center

<https://docs.mcapilot.com/en/articles/12081782-how-to-match-funders-automatically-for-a-deal>

36 37 94 166 168 169 172 174 AI Scan Applications | MCA Pilot Help Center

<https://docs.mcapilot.com/en/articles/12614992-ai-scan-applications>

40 41 42 43 44 45 48 49 63 71 72 AI Scan Criteria: Automatically Extract Funder Criteria from PDFs | MCA Pilot Help Center

<https://docs.mcapilot.com/en/articles/13182280-ai-scan-criteria-automatically-extract-funder-criteria-from-pdfs>

52 53 58 60 How to Group Funders for Submissions | MCA Pilot Help Center

<https://docs.mcapilot.com/en/articles/10743139-how-to-group-funders-for-submissions>

67 68 Features | MCA Pilot Help Center

<https://docs.mcapilot.com/en/collections/10769624-features>

85 umsousercontent.com

https://umsousercontent.com/lib_wwgfWmsCPwcuaYx/ej8rn1rjokd6udj3.png?w=420&dpr=2

121 umsousercontent.com

https://umsousercontent.com/lib_wwgfWmsCPwcuaYx/ekn31nb3q8mcy0y.png?w=630&dpr=2

131 132 134 135 137 138 139 140 143 144 How to add commissions to an Advance | MCA Pilot Help Center

<https://docs.mcapilot.com/en/articles/10895297-how-to-add-commissions-to-an-advance>

133 umsousercontent.com

https://umsousercontent.com/lib_wwgfWmsCPwcuaYx/lad58mfs2kk6hljg.png?w=420&dpr=2

141 142 159 How to Add Reverse Consolidations and Schedule Weekly Distributions | MCA Pilot Help Center

<https://docs.mcapilot.com/en/articles/12110845-how-to-add-reverse-consolidations-and-schedule-weekly-distributions>

167 How to Add Funders into MCA Pilot | MCA Pilot Help Center

<https://docs.mcapilot.com/en/articles/12710149-how-to-add-funders-into-mca-pilot>

170 171 The Only All-in-One MCA Platform - NextLevel MCA

<https://nextlevelmca.com/vs>